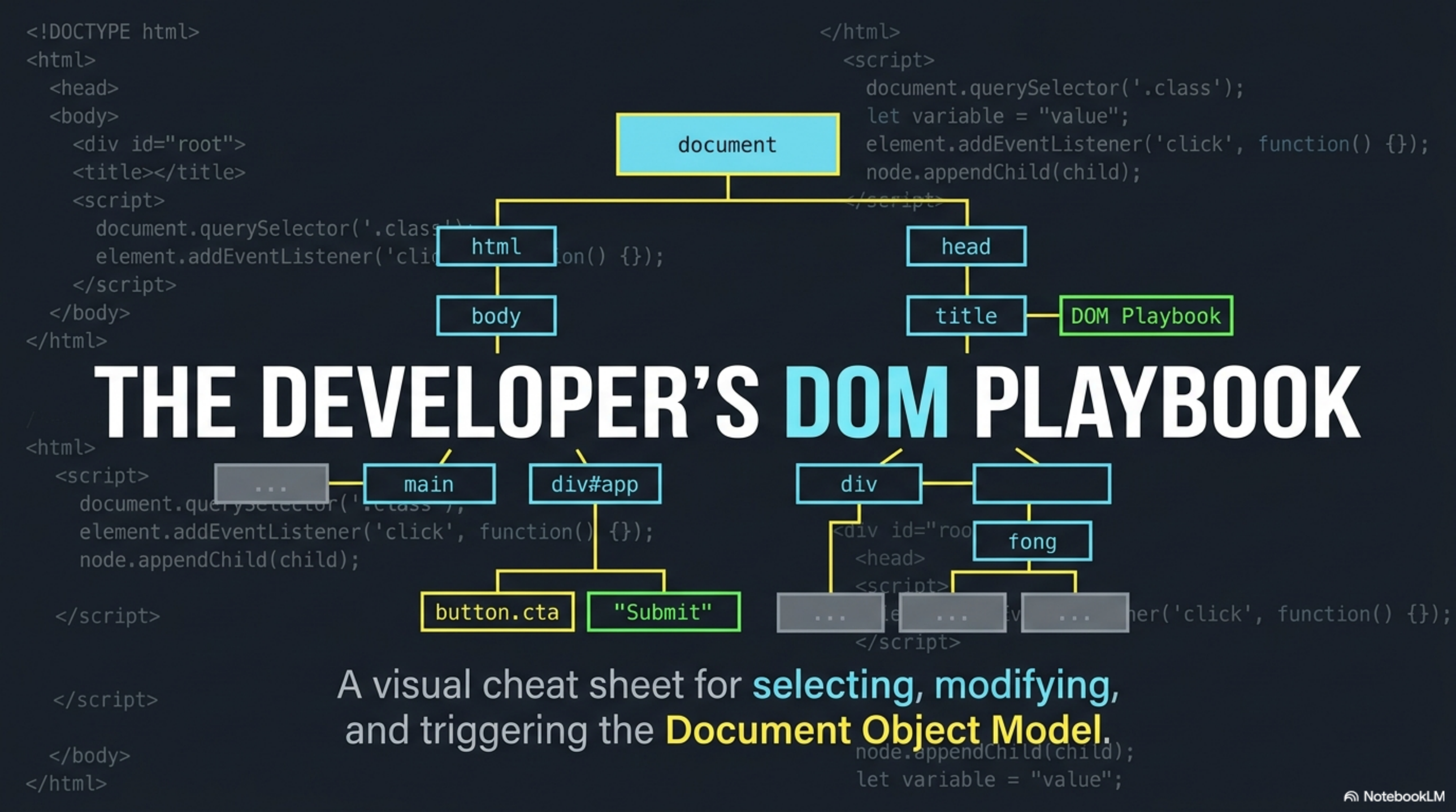
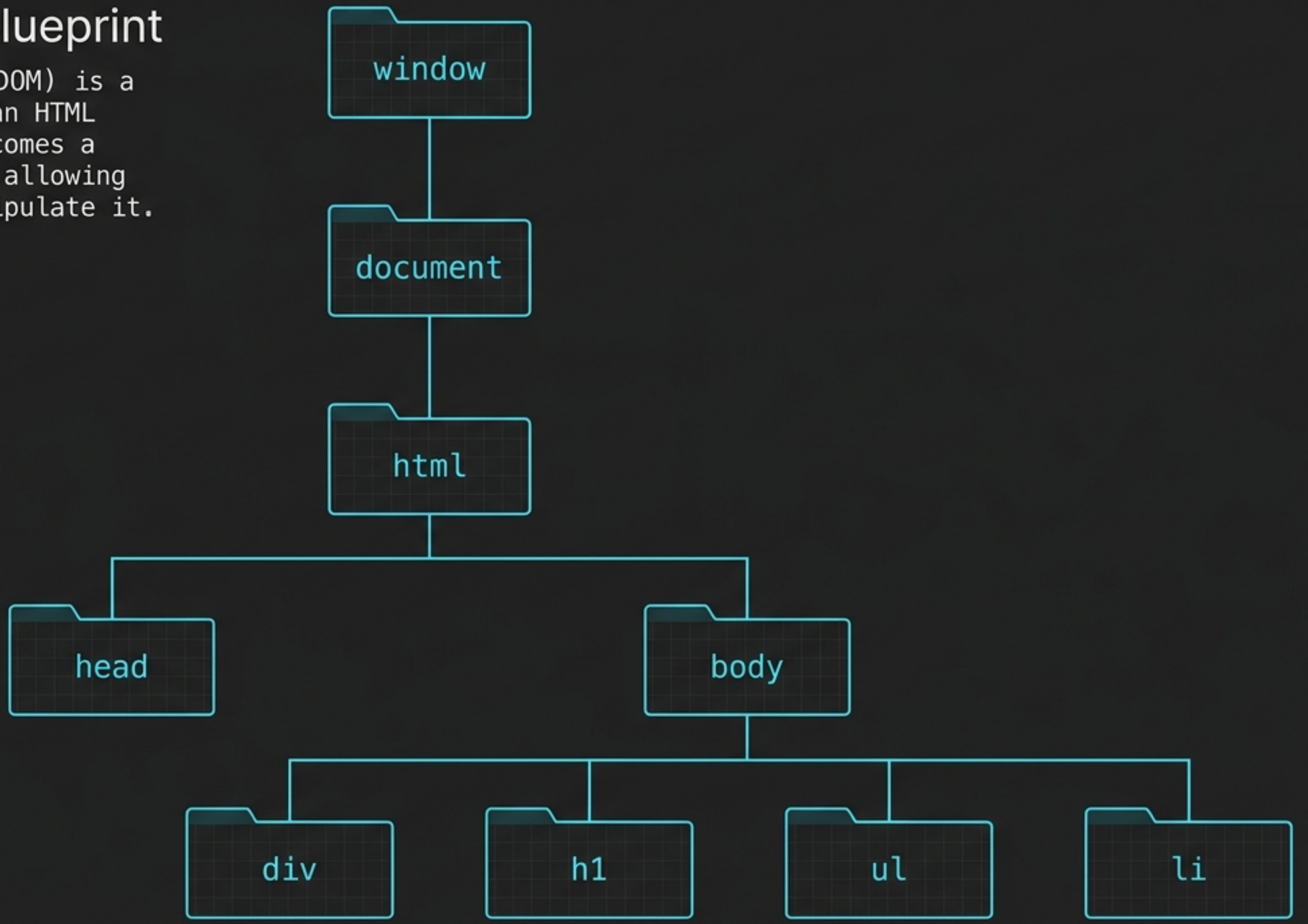


THE DEVELOPER'S DOM PLAYBOOK

A visual cheat sheet for selecting, modifying, and triggering the Document Object Model.

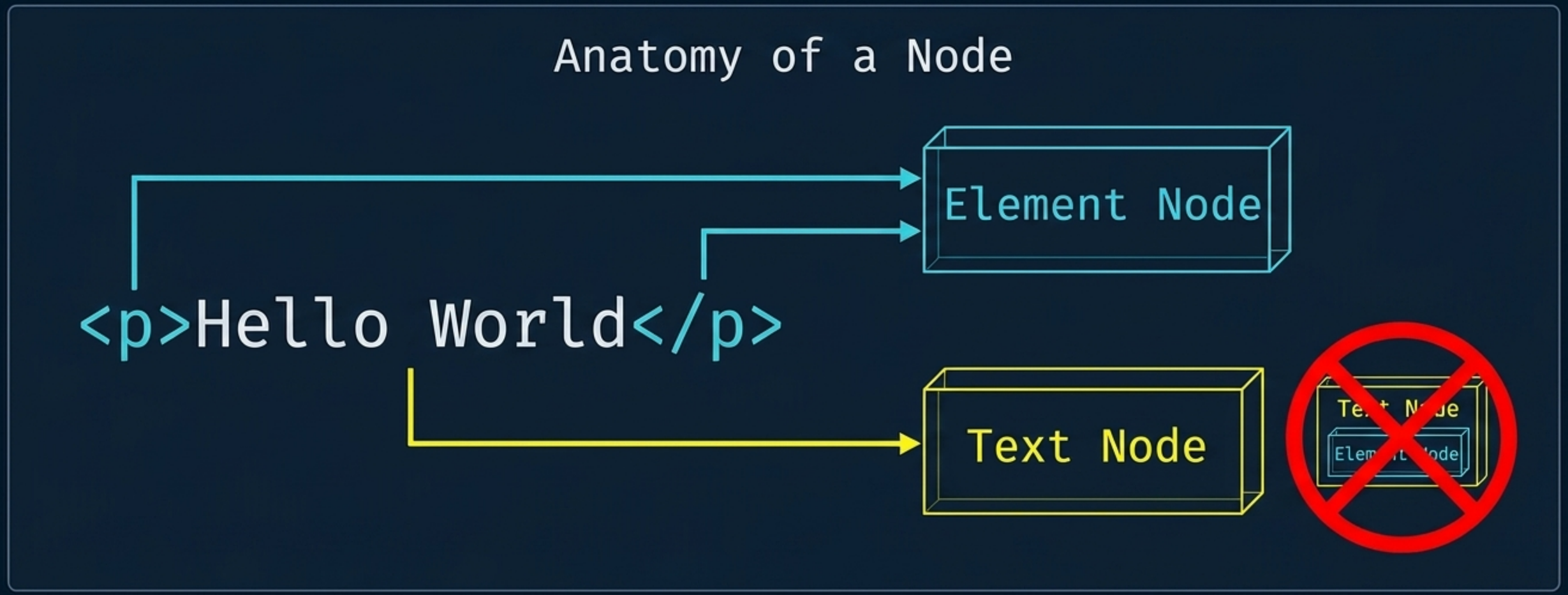
The Web Page is a Blueprint

The Document Object Model (DOM) is a tree-like representation of an HTML page. Every tag you write becomes a "node" in this architecture, allowing JavaScript to target and manipulate it.



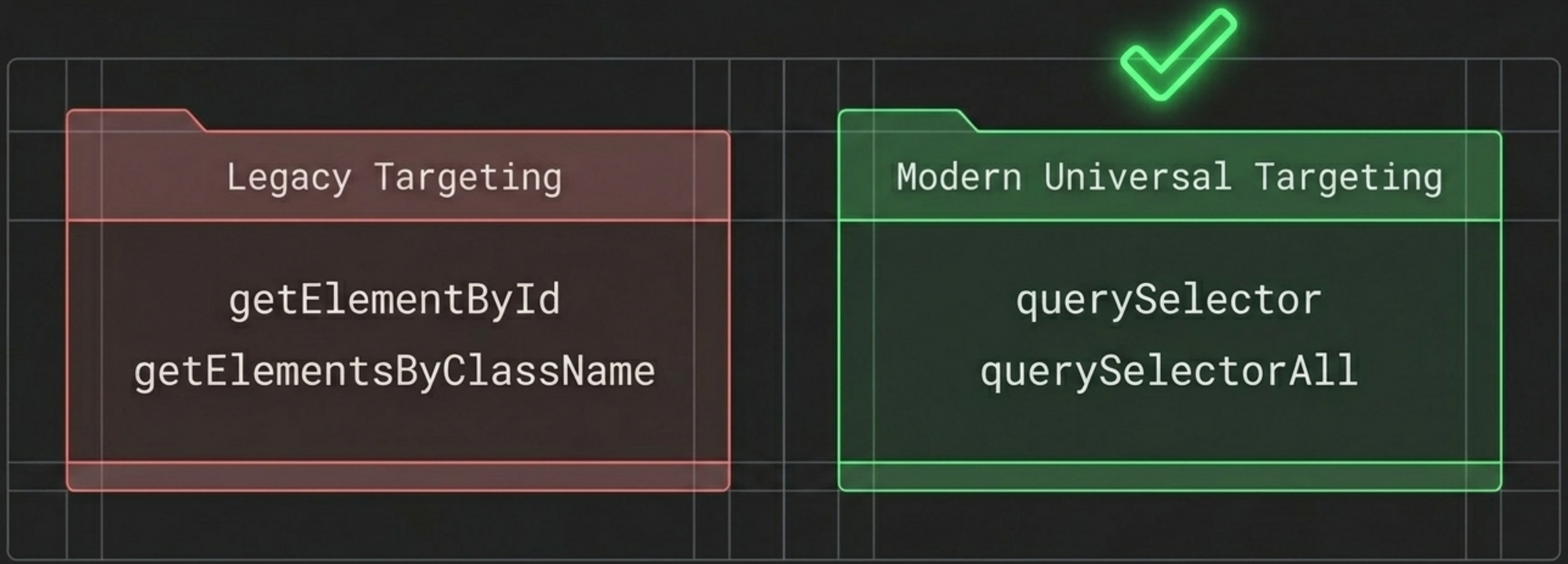
Elements vs. Text Nodes

An Element Node is the actual HTML container (the tag). A Text Node is the pure content inside it. Text nodes are terminal—they can never contain child nodes.

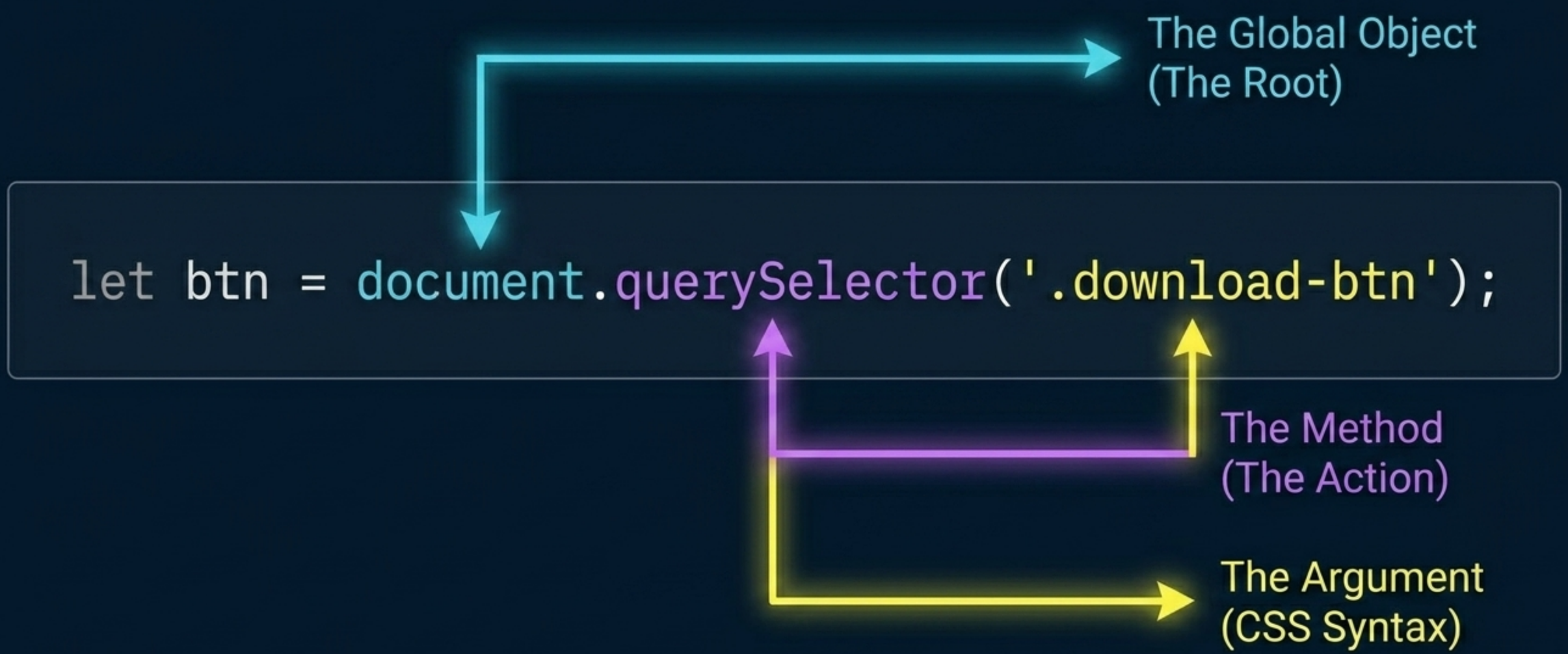


Pinpointing Elements in the Tree

You cannot manipulate what you cannot select. While legacy selectors still exist, modern workflows rely on universal selectors that accept standard CSS syntax for maximum precision.



Anatomy of a Selector



The Selection Matrix

Selector	Syntax Example	Return Type	Notes
`getElementById`	`('header')`	Single Element	Fast, but restricted to ID only.
`getElementsByClassName`	`('card')`	`HTMLCollection`	Returns an array-like structure.
`querySelector`	`('.card')`	Single Element	Returns only the *first* match.
`querySelectorAll`	`('li.item')`	`NodeList`	Returns all matches; allows forEach looping.

Peeking Under the Hood

Use `console.dir()` instead of `console.log()` to view an element as an interactive JavaScript object. This reveals the hidden properties (like `.style` or `.classList`) available for manipulation.

Console Output

```
> console.log(h1)
  <h1>Hey</h1>
```

Console Output

```
> console.dir(h1)
  ▼ h1 {
    ▶ align: "", attributes: NamedNodeMap, baseURI: "...",
    ▶ classList: DOMTokenList, style: CSSStyleDeclaration... }
```

The Content Modification Matrix

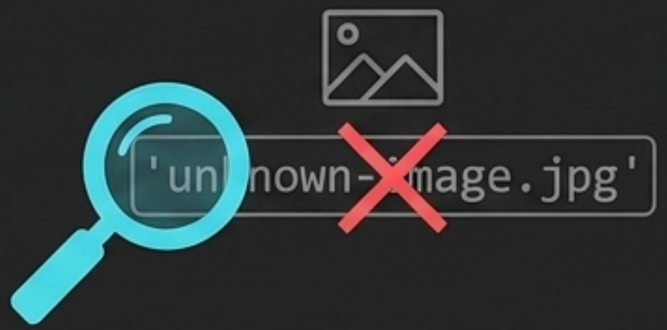
innerText	textContent	innerHTML
		
Reads/writes only the visible text on the screen. Slower.	Reads/writes all text, even if hidden by display: none. Faster (Preferred)	Parses and renders raw HTML tags (e.g., Hey becomes italicized).

Manipulating Element Attributes

Attributes (like `src`, `href`, or `disabled`) are the metadata of an element. JavaScript allows you to read, inject, or entirely erase this metadata on the fly.

Get:

```
img.getAttribute('src')
```

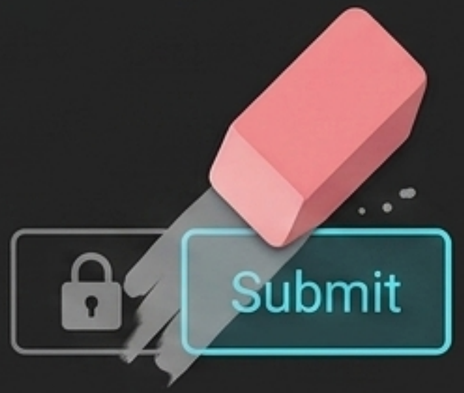


Set:

```
img.setAttribute('src',
'new-image.jpg')
```

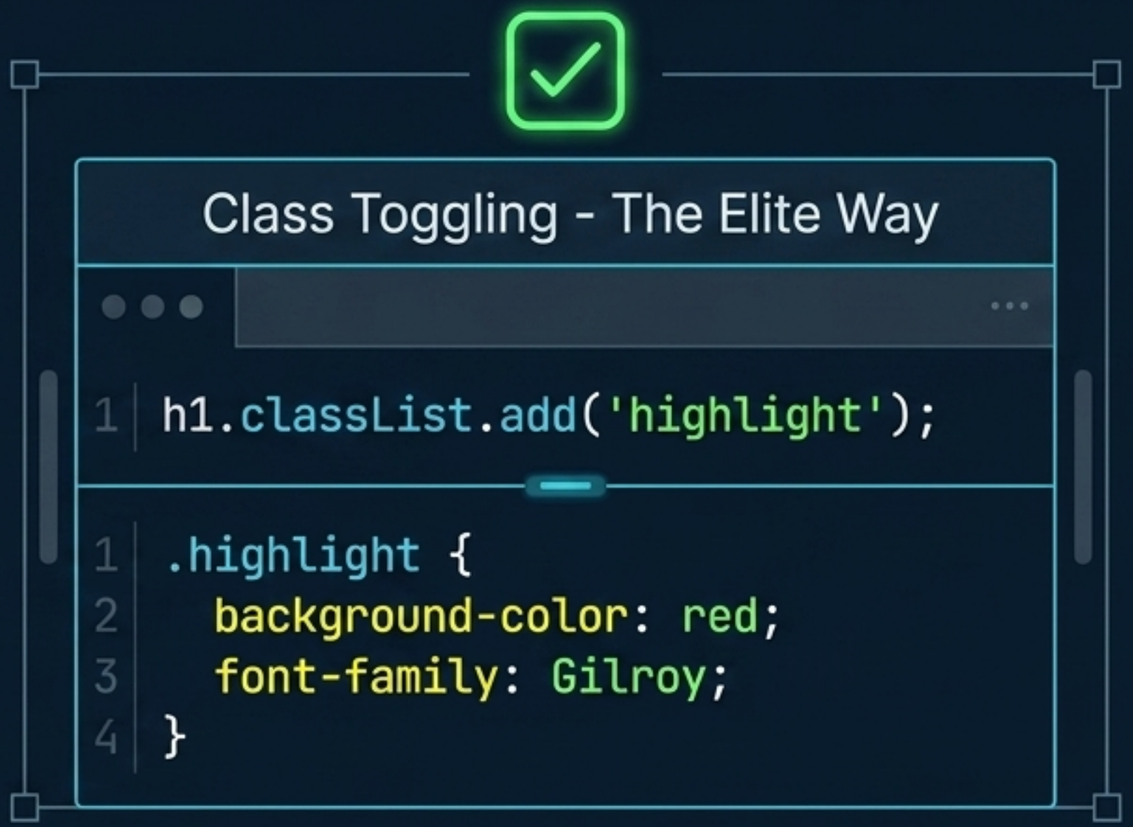
Remove:

```
img.removeAttribute(
'disabled')
```



Styling: Inline vs. Class Injection

Injecting inline styles alters CSS directly using camelCase syntax. However, the best practice is pre-writing CSS classes and using `.classList` to toggle them.



The Toggle Mechanism

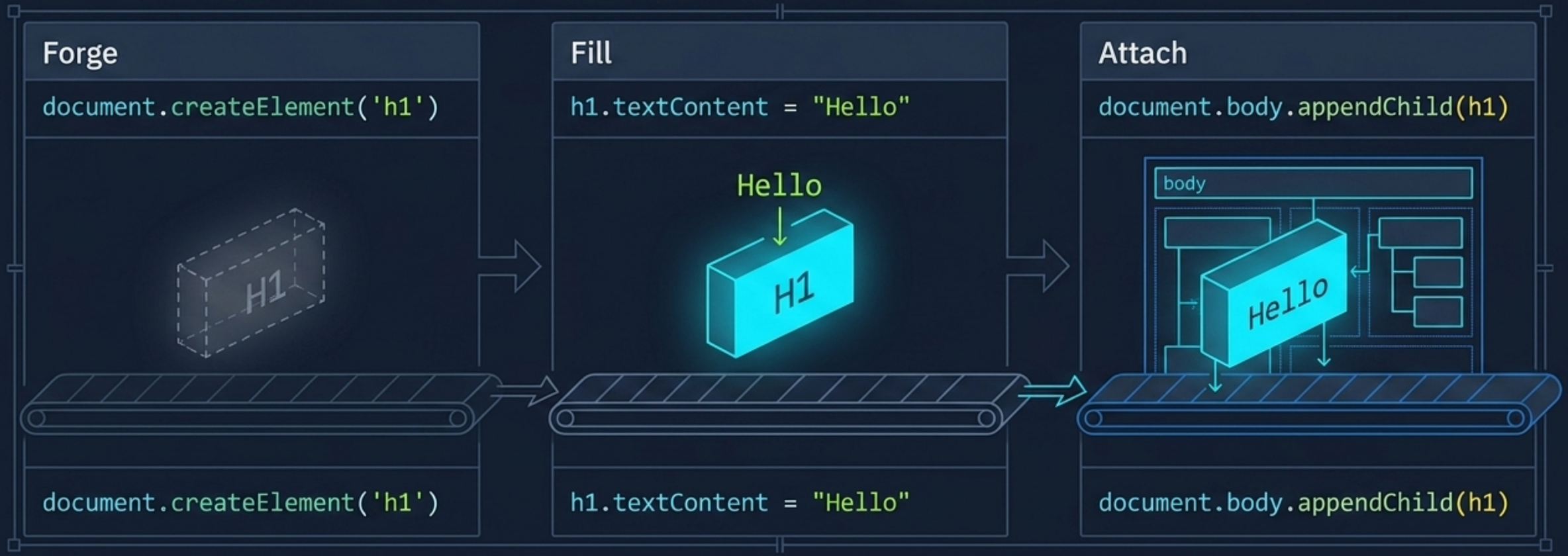
The `.toggle()` method acts as a smart switch. If the class exists, it removes it. If the class is missing, it adds it.

`.classList.add('active')` (forces ON)

`.classList.remove('active')` (forces OFF)

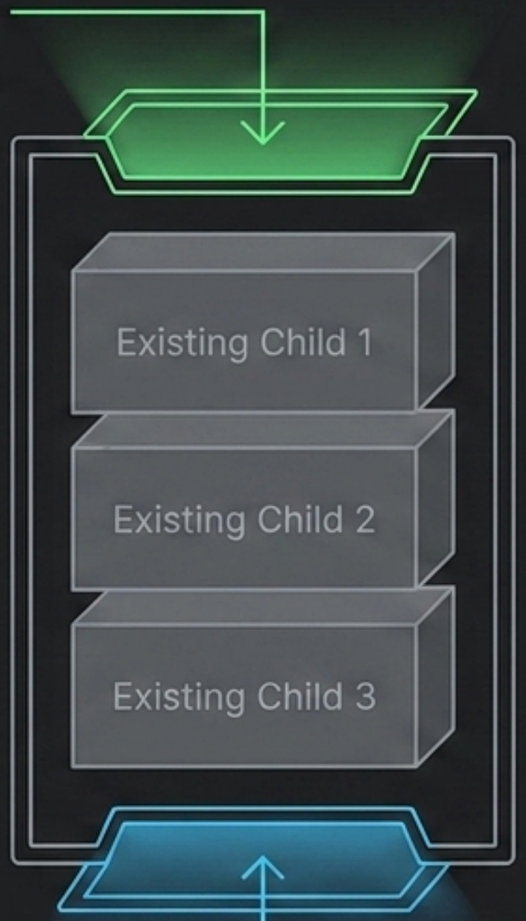
The Element Assembly Line

Creating a DOM element happens in a void. It remains invisible until you give it content and explicitly attach it to an existing parent node in the tree.



Insertion Points: Prepend vs. Append

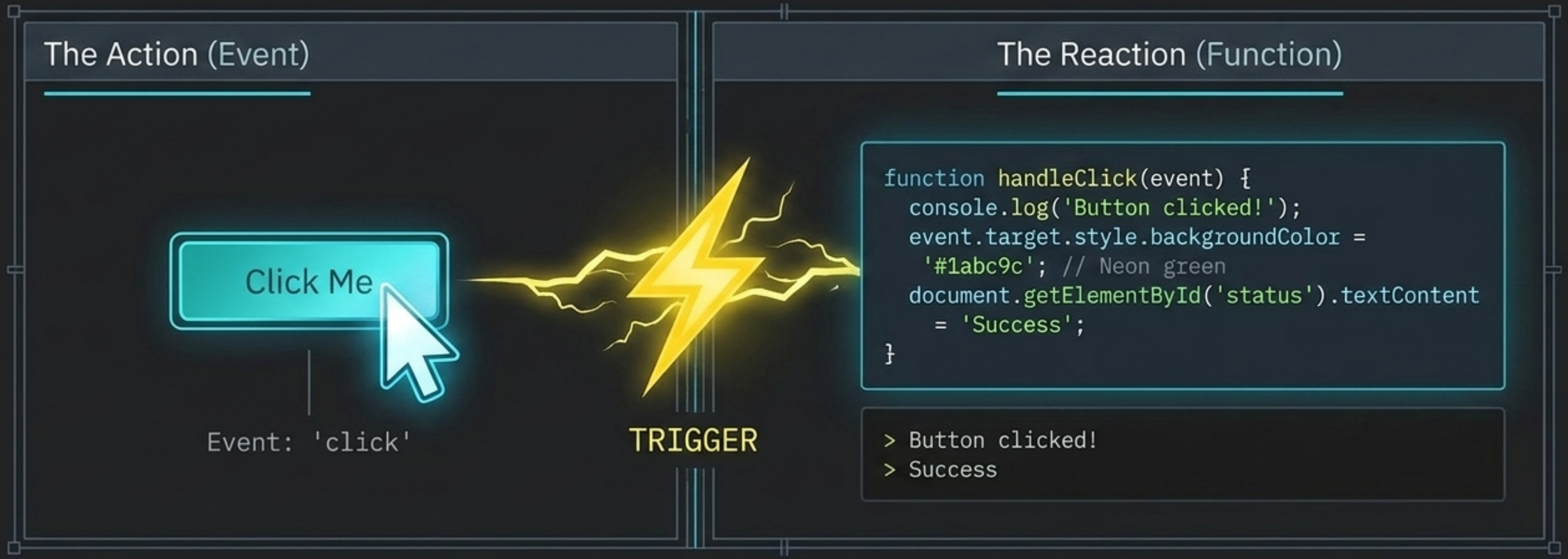
`prepend(element)`
inserts a block *_before_* all existing content.



`appendChild(element)`
inserts a block *_after_* all existing content.

Making the DOM Interactive

An Event Listener bridges the gap between user behavior and code execution.
You define the trigger (the event) and provide the payload (the function).



Anatomy of an Event Listener

The Target
(Who is listening?)

```
btn.addEventListener('click', function() { … });
```

The Event
(What are we listening for?)

The Callback
(What happens when
it triggers?)

The Core Event Triggers

Mouse

`click`

`dblclick`

Keyboard

`input`

Fires continuously on every keystroke.

Form

`change`

Fires when a selection is finalized.

`submit`

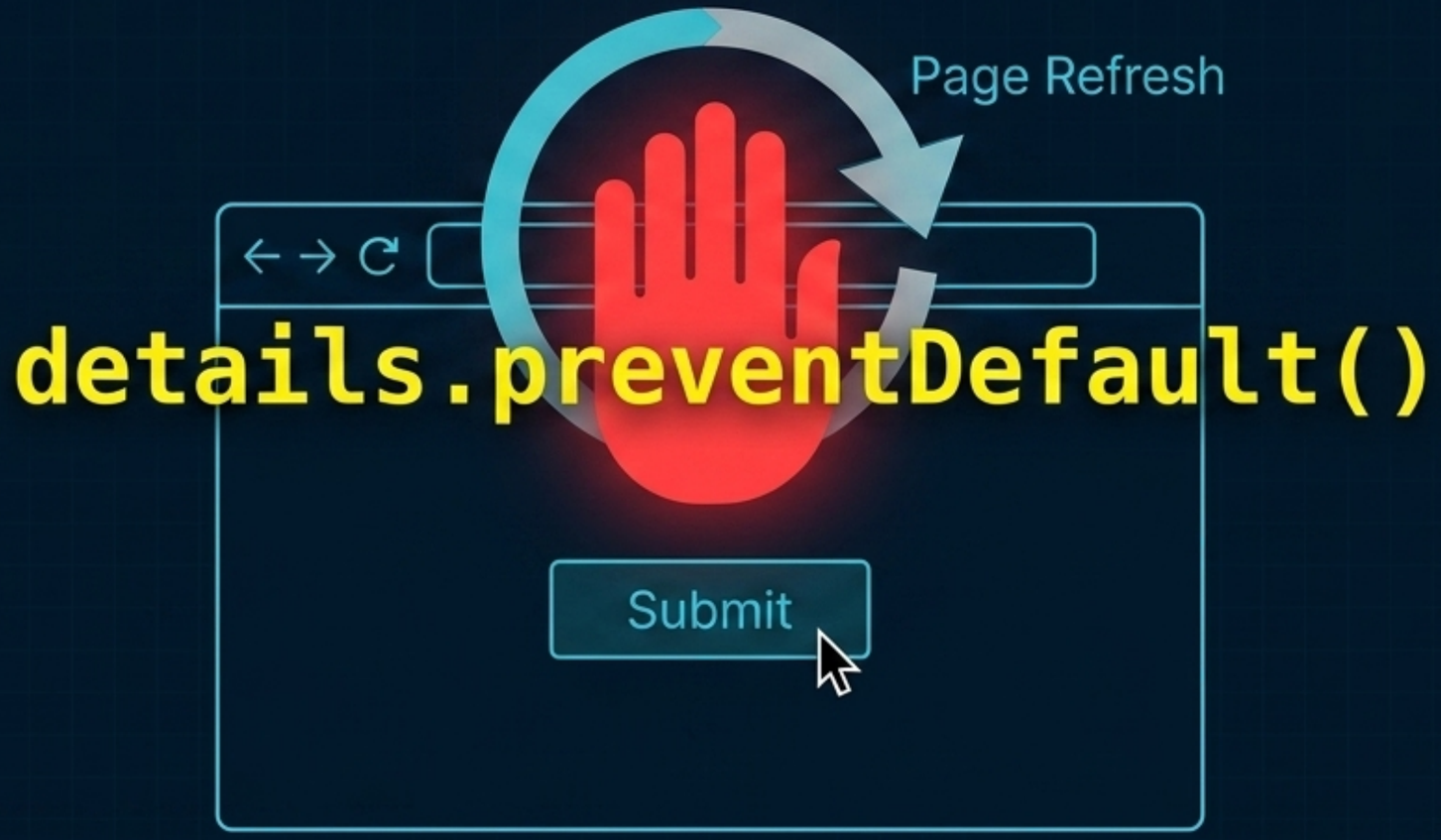
Pinpointing the Target

The event function receives a hidden payload containing details about the action. `details.target` reveals the exact element the user interacted with, enabling highly specific manipulations.

```
function(
  details
) {
  console.log(
    details.target
  );
}
```

Halting Default Behaviors

Browsers have default behaviors—like reloading the entire page when a form is submitted. Using `preventDefault()` inside your listener stops this, allowing you to handle the data dynamically via JavaScript.



The Event Bubbling Sonar

If an event occurs on an element (like a click) and it has no listener, the event “bubbles” upward, triggering listeners on its parent, then its grandparent, all the way up the DOM tree.

The DOM Manipulation Workflow

The DOM is not a static document. It is a continuous, interconnected feedback loop between the user's actions and the browser's dynamic architecture.